

Spatial Regularization in Federated Learning: Predicting Temperature via Network Consensus

Balint Takacs

May 17, 2026

1 Introduction and Motivation

In real-world Federated Learning (FL) applications, individual clients—in this case, weather stations—often possess limited, noisy, or geographically sparse data. When a single weather station attempts to train a machine learning model using only its own small dataset, the model is highly susceptible to overfitting and typically generalizes poorly to unseen data.

This project tackles the challenge of data scarcity by connecting weather stations into a spatial graph. By leveraging the Graph Total Variation Minimization (GTVMin) framework, we impose a structural prior on the learning process: neighboring stations (nodes connected by edges) should learn local models that are structurally similar. This injects the physical domain knowledge that weather patterns are spatially continuous, allowing individual stations to collaborate without centralizing their raw data.

2 Problem Formulation

This section formally maps the physical weather prediction problem into the Federated Learning framework, defining the data structures, network configuration, and the learning objective for individual stations. The network is modeled over an empirical graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, A)$.

2.1 Data and Task

- **Task:** Predicting Air Temperature based on local meteorological factors.
- **Data Range:** January 1, 2024 to February 1, 2024.
- **Features (x):** Derived from available meteorological readings including Cloud amount, Dew-point temperature, Gust speed, Horizontal visibility, Precipitation amount, Precipitation intensity, Present weather (auto), Pressure (msl), Relative humidity, Snow depth, Wind direction, and Wind speed. The data structure also records station name, FMI ID, spatial coordinates (latitude/longitude), timestamps, and measurement units.

- **Label (y):** Air temperature.
- **Local Dataset:** Each station i has a local dataset of m_i readings:
 $\mathcal{D}^{(i)} = \{(\mathbf{x}_1^{(i)}, y_1^{(i)}), \dots, (\mathbf{x}_{m_i}^{(i)}, y_{m_i}^{(i)})\}.$

2.2 Network Topology and Models

- **Nodes ($\mathcal{V}$):** Each node $i \in \mathcal{V}$ represents an FMI weather station.
- **Edges ($\mathcal{E}$):** Defined by the Delaunay Triangulation of the stations' geographic coordinates (longitude, latitude). The Delaunay Triangulation creates a non-overlapping mesh of triangles connecting the points such that no point lies inside the circum-circle of any triangle. This mathematical property is needed because it naturally connects adjacent neighbor stations without relying on arbitrary, hard-coded distance thresholds.
- **Edge Weights ($A_{i,j}$):** Determined by the Haversine distance between stations. For an edge between station i and j , $A_{i,j} = \max(0, 60 - \text{dist}(i, j))$. This ensures that only local climatic neighborhoods influence each other, degrading linearly to 0 at 60km.
- **Hypothesis Class and Loss:** We use Linear Regression. For each station i , the model is parameterized by a weight vector $\mathbf{w}^{(i)}$, with predictions $\hat{y} = (\mathbf{w}^{(i)})^T \mathbf{x}$. The local loss $L_i(\mathbf{w}^{(i)})$ is the Mean Squared Error (MSE) computed on dataset $\mathcal{D}^{(i)}$.

3 Methodology

This section details the mathematical core of the network consensus model. We define the global graph objective, derive the distributed local update protocol (the fixed-point equation), and outline the data augmentation strategy used for computational implementation.

3.1 Global Objective

We aim to find the optimal set of models for all stations, $\mathbf{w} = [\mathbf{w}^{(1)}, \mathbf{w}^{(2)}, \dots, \mathbf{w}^{(n)}]$, by minimizing the following global objective function $J(\mathbf{w})$:

$$J(\mathbf{w}) = \sum_{i=1}^n L_i(\mathbf{w}^{(i)}) + \alpha \sum_{(i,j) \in \mathcal{E}} A_{i,j} \|\mathbf{w}^{(i)} - \mathbf{w}^{(j)}\|_2^2 \quad (1)$$

The first term ensures fidelity to local data, while the second term (GTV penalty) enforces consensus among neighboring models. The regularization parameter α controls this trade-off.

3.2 Fixed-Point Equation

From the perspective of a single node i , minimizing its local cost function while holding its neighbors' models constant yields a local cost $J_i(\mathbf{w}^{(i)})$. Let the degree of node i be

$d_i = \sum_{j \in \mathcal{N}(i)} A_{i,j}$, and define the aggregate "message" received from neighbors as:

$$\bar{\mathbf{w}}^{(i)} = \frac{1}{d_i} \sum_{j \in \mathcal{N}(i)} A_{i,j} \mathbf{w}^{(j)} \quad (2)$$

The local cost simplifies to:

$$J_i(\mathbf{w}^{(i)}) = L_i(\mathbf{w}^{(i)}) + \alpha \cdot d_i \|\mathbf{w}^{(i)} - \bar{\mathbf{w}}^{(i)}\|_2^2 + \text{Constant} \quad (3)$$

At iteration $t + 1$, node i updates its model by evaluating the fixed-point operator:

$$\mathbf{w}^{(i,t+1)} = \arg \min_{\mathbf{w}} (L_i(\mathbf{w}) + \alpha \cdot d_i \|\mathbf{w} - \bar{\mathbf{w}}^{(i,t)}\|_2^2) \quad (4)$$

By taking the derivative of this objective with respect to $\mathbf{w}$ and setting it to zero, we arrive at the explicit fixed-point equation for the local update:

$$\mathbf{w}^{(i,t+1)} = (X_i^T X_i + \alpha \cdot d_i I)^{-1} (X_i^T y_i + \alpha \cdot d_i \bar{\mathbf{w}}^{(i,t)}) \quad (5)$$

where X_i and y_i are the data matrix and label vector for station i , respectively.

3.3 Implementation strategy

This optimization mimics Ridge Regression centered around $\bar{\mathbf{w}}^{(i,t)}$. We implement this via data augmentation, augmenting the local training features X_i with $\sqrt{\alpha \cdot d_i} \cdot I$, and the labels y_i with $\sqrt{\alpha \cdot d_i} \cdot \bar{\mathbf{w}}^{(i,t)}$. The update is then solved using standard Ordinary Least Squares (OLS) solvers (e.g., provided by `scikit-learn` [1]).

4 Numerical Experiments (Draft Plan)

This section will document the numerical validation, experimental setup, and the structural baselines used to prove the efficacy of the proposed spatial regularization approach.

This section will document the numerical validation of our approach.

- **Data Splitting:** The time-series data will be chronologically divided into Train (60%), Validation (20%), and Test (20%) sets.
- **Baselines:** We will compare the proposed FL GTVMin method against two structural baselines:
 1. *Local-only Training* ($\alpha = 0$): Independent local models with no communication.
 2. *Global Shared Model* ($\alpha \rightarrow \infty$): A single model trained on pooled data. To ensure a fair comparison, this global model will interdependently have its own L_2 (Ridge) penalty tuned via a separate hyperparameter search.
- **Hyperparameter Tuning:** Model selection for the optimal α will be performed over a log-scale grid evaluating on the Validation MSE.

5 Reproducibility

To ensure reproducibility and transparency, the final report will act as a complete manifest for the experiments, detailing the exact data and hyperparameter configurations used.

To ensure reproducibility, the final report will include:

- The exact date range of the FMI data downloaded (January 1, 2024 to February 1, 2024).
- The specific structural features selected from the 13 downloaded variables for the regression task.
- The optimal α discovered during validation.
- All random seeds initialization parameters (if applicable).

References

- [1] Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.